IoT Lab Programs

Study Material — Practical Exam Ready

No.	Program Title
1	Arduino Installation and Blink LED using Node MCU Esp8266
2	Interfacing IR (Infrared Proximity) Sensor with Node MCU Esp8266
3	Interfacing Light Sensor with Node MCU Esp8266
4	Interfacing Temperature Humidity Sensor with Node MCU Esp8266
5	Interfacing Soil Moisture Sensor with Node MCU Esp8266
6	Interfacing MQ4 Gas Sensor with Node MCU Esp8266
7	Interfacing Pulse Sensor with Node MCU Esp8266
8	Sound Sensor
9	Ultrasonic Sensor

Program 1: Arduino Installation and Blink LED using Node MCU Esp8266

AIM

To install the Arduino IDE, configure it for the NodeMCU ESP8266 board, and write a program to blink an LED connected to pin D4.

PROCEDURE

1. Download and install Arduino IDE from www.Arduino.cc.
2. Connect NodeMCU ESP8266 to the computer using a Micro USB cable.
3. Open Arduino IDE → File → Preferences → Settings.
4. Paste the following URL in Additional Board Manager URL:

http://arduino.esp8266.com/stable/package_esp8266com_index.json

5. Go to Tools → Boards → Board Manager. Search 'esp8266' and install 'esp8266 by esp8266 Community'.
6. Go to Tools → Boards → Select NodeMCU 1.0 (ESP-12E Module).
7. Go to Tools → Port → Select COM3 (install CP210x drivers if COM3 is unavailable).
8. Connect LED +ve pin to D4 and LED -ve pin to GND on NodeMCU.
9. Write and upload the Arduino script given below.
10. Observe the LED blinking ON and OFF every 1 second.

SOURCE CODE

```
void setup() {
  pinMode(D4, OUTPUT); // Set D4 as output
}

void loop() {
  digitalWrite(D4, HIGH); // Turn ON LED
  delay(1000);
  digitalWrite(D4, LOW);  // Turn OFF LED
  delay(1000);
}
```

RESULT

The program was successfully uploaded to NodeMCU ESP8266. The LED connected to pin D4 blinks ON and OFF at 1-second intervals as expected.

Program 2: Interfacing IR (Infrared Proximity) Sensor with Node MCU Esp8266

AIM

To interface an IR (Infrared Proximity) Sensor with NodeMCU ESP8266 and control an LED based on obstacle detection.

PROCEDURE

1. Connect IR sensor VCC pin to 3.3V pin on NodeMCU.
2. Connect IR sensor GND pin to GND on NodeMCU.
3. Connect IR sensor OUT pin to D0 on NodeMCU.
4. Connect LED +ve pin to D5 and LED -ve pin to GND.
5. Open Arduino IDE and select NodeMCU 1.0 board and correct COM port.
6. Write and upload the given Arduino script.
7. When an object is detected by the IR sensor (output LOW), the LED on D5 turns ON.
8. When no object is detected (output HIGH), the LED turns OFF.
9. Open Serial Monitor at 9600 baud to observe sensor values.

SOURCE CODE

```
void setup() {
  pinMode(D0, INPUT);  // Sensor input
  pinMode(D5, OUTPUT); // Output pin
  Serial.begin(9600);  // Initialize serial communication
}

void loop() {
  int sensorValue = digitalRead(D0);
  Serial.println(sensorValue);
  if (sensorValue < 1) {
    digitalWrite(D5, HIGH); // Turn ON output
  } else {
    digitalWrite(D5, LOW);  // Turn OFF output
  }
  delay(100); // Add a small delay for stability
}
```

RESULT

The IR sensor was successfully interfaced with NodeMCU ESP8266. The LED turns ON when an obstacle is detected and turns OFF when no obstacle is present.

Program 3: Interfacing Light Sensor (LDR) with Node MCU Esp8266

AIM

To interface an LDR (Light Dependent Resistor) light sensor with NodeMCU ESP8266 and control an LED based on light intensity.

PROCEDURE

1. Connect LDR sensor VCC pin to 3.3V pin on NodeMCU.
2. Connect LDR sensor GND pin to GND on NodeMCU.
3. Connect LDR sensor A0 pin to A0 on NodeMCU.
4. Connect LED +ve pin to D5 and LED -ve pin to GND.
5. Open Arduino IDE and select NodeMCU 1.0 board and correct COM port.
6. Write and upload the given Arduino script.
7. When light is detected (sensor value HIGH), LED on D5 turns ON.
8. When darkness is detected (sensor value LOW), LED turns OFF.
9. Open Serial Monitor at 9600 baud to observe real-time sensor values.

SOURCE CODE

```
void setup() {
  pinMode(D0, INPUT);  // Sensor input
  pinMode(D5, OUTPUT); // Output pin
  Serial.begin(9600);  // Initialize serial communication
}

void loop() {
  int sensorValue = digitalRead(D0); // Read sensor value
  Serial.println(sensorValue);
  if (sensorValue < 1) {
    digitalWrite(D5, LOW);  // Turn OFF output
  } else {
    digitalWrite(D5, HIGH); // Turn ON output
  }
  delay(100); // Small delay for stability
}
```

RESULT

The LDR light sensor was successfully interfaced with NodeMCU ESP8266. The LED turns ON in the presence of light and turns OFF in darkness.

Program 4: Interfacing Temperature & Humidity Sensor (DHT11) with Node MCU Esp8266

AIM

To interface a DHT11 Temperature and Humidity Sensor with NodeMCU ESP8266 and display temperature and humidity readings on the Serial Monitor.

PROCEDURE

1. Connect DHT11 VCC pin to 3.3V pin on NodeMCU.
2. Connect DHT11 GND pin to GND on NodeMCU.
3. Connect DHT11 Data pin (D0) to D1 on NodeMCU (as defined in code).
4. Connect LED +ve pin to D5 and LED -ve pin to GND.
5. Open Arduino IDE. Install the 'DHT sensor library' via Sketch → Include Library → Manage Libraries.
6. Select NodeMCU 1.0 board and correct COM port.
7. Write and upload the given Arduino script.
8. Open Serial Monitor at 9600 baud rate.
9. Temperature (in °C) and Humidity (%) readings will be printed every 1 second.

SOURCE CODE

```
#include "DHT.h"
#include <ESP8266WiFi.h>

#define DHTTYPE DHT11
const int DHTPin = D1;
DHT dht(DHTPin, DHTTYPE);

void setup() {
  Serial.begin(9600);
  delay(500);
  dht.begin();
  delay(500);
}

void loop() {
  int h = dht.readHumidity();
  float t = dht.readTemperature();

  String Temp = "Temperature: " + String(t) + " C";
  String Humi = "Humidity: " + String(h) + "%";

  Serial.println(Temp);
  Serial.println(Humi);
  delay(1000);
```

```
}
```

RESULT

The DHT11 sensor was successfully interfaced with NodeMCU ESP8266. Temperature and humidity values are displayed accurately on the Serial Monitor every second.

Program 5: Interfacing Soil Moisture Sensor with Node MCU Esp8266

AIM

To interface a Soil Moisture Sensor with NodeMCU ESP8266 and monitor soil moisture percentage, turning on an LED alert when moisture is critically low.

PROCEDURE

1. Connect Soil Moisture Sensor VCC pin to 3.3V pin on NodeMCU.
2. Connect Soil Moisture Sensor GND pin to GND on NodeMCU.
3. Connect Soil Moisture Sensor A0 pin to A0 on NodeMCU.
4. Connect LED +ve pin to D5 and LED -ve pin to GND.
5. Open Arduino IDE and select NodeMCU 1.0 board and correct COM port.
6. Write and upload the given Arduino script.
7. Insert the soil moisture sensor probes into soil.
8. Open Serial Monitor at 9600 baud to observe moisture percentage.
9. If moisture percentage is <= 10%, LED turns ON (relay ON state = LOW) indicating dry soil.
10. If moisture is above 10%, LED turns OFF.

SOURCE CODE

```
#include <ESP8266WiFi.h>

const byte relayOnState  = LOW;
const byte relayOffState = HIGH;

int SoilMoist = A0; // Soil moisture sensor connected to A0
int LED = D5;       // LED connected to D5

void setup() {
  pinMode(SoilMoist, INPUT); // Set SoilMoist sensor as input
  pinMode(LED, OUTPUT);       // Set LED as output
  Serial.begin(9600);         // Initialize serial communication
}

void loop() {
  float moisture_percentage;
  // Calculate moisture percentage
  moisture_percentage = (100.00 - ((analogRead(SoilMoist) / 1023.00) * 100.00));
  Serial.println(moisture_percentage); // Print moisture percentage
  if (moisture_percentage <= 10) {
    digitalWrite(LED, relayOnState);  // Turn ON LED (LOW)
  } else {
```

```
      digitalWrite(LED, relayOffState); // Turn OFF LED (HIGH)
    }
    delay(1000); // Wait 1 second before next reading
  }
```

RESULT

The Soil Moisture Sensor was successfully interfaced with NodeMCU ESP8266. Moisture percentage is displayed on the Serial Monitor, and the LED activates when soil moisture falls below 10%.

Program 6: Interfacing MQ4 Gas Sensor with Node MCU Esp8266

AIM

To interface an MQ4 Gas Sensor with NodeMCU ESP8266 and trigger a buzzer alarm when gas/smoke concentration exceeds a defined threshold.

PROCEDURE

1. Connect MQ4 Gas Sensor VCC pin to 3.3V pin on NodeMCU.
2. Connect MQ4 Gas Sensor GND pin to GND on NodeMCU.
3. Connect MQ4 Gas Sensor A0 pin to A0 on NodeMCU.
4. Connect Buzzer +ve pin to D2 and Buzzer -ve pin to GND.
5. Open Arduino IDE and select NodeMCU 1.0 board and correct COM port.
6. Write and upload the given Arduino script.
7. Open Serial Monitor at 9600 baud to observe real-time sensor values.
8. When gas/smoke sensor value exceeds 500 (threshold), buzzer turns ON.
9. When sensor value is below threshold, buzzer remains OFF.

SOURCE CODE

```
#include <ESP8266WiFi.h>

#define smokeA0 A0 // Smoke sensor connected to A0
#define Alarm   D2 // Alarm connected to D2

int sensorThres = 500; // Threshold value for smoke detection

void setup() {
  pinMode(Alarm,   OUTPUT); // Set Alarm as output
  pinMode(smokeA0, INPUT);  // Set Smoke Sensor as input
  Serial.begin(9600);        // Start serial communication
}

void loop() {
  int sensorValue = analogRead(smokeA0); // Read smoke sensor value
  Serial.print("Sensor Value: ");
  Serial.println(sensorValue);           // Print sensor value

  if (sensorValue > sensorThres) {
    digitalWrite(Alarm, HIGH); // Turn ON alarm
  } else {
    digitalWrite(Alarm, LOW);  // Turn OFF alarm
  }
  delay(100); // Short delay for stability
```

```
}
```

RESULT

The MQ4 Gas Sensor was successfully interfaced with NodeMCU ESP8266. The buzzer alarm triggers when gas concentration exceeds the threshold of 500 and deactivates when the level drops below it.

Program 7: Interfacing Pulse Sensor with Node MCU Esp8266

AIM

To interface a Pulse Sensor with NodeMCU ESP8266 to detect heartbeat signals and turn on an LED when a heartbeat is detected above a threshold.

PROCEDURE

1. Connect Pulse Sensor VCC pin to 3.3V pin on NodeMCU.
2. Connect Pulse Sensor GND pin to GND on NodeMCU.
3. Connect Pulse Sensor A0 (signal) pin to A0 on NodeMCU.
4. Connect LED +ve pin to D5 and LED -ve pin to GND.
5. Open Arduino IDE and select NodeMCU 1.0 board and correct COM port.
6. Write and upload the given Arduino script.
7. Place the pulse sensor on a fingertip.
8. Open Serial Monitor / Serial Plotter at 9600 baud to observe pulse waveform.
9. When the signal exceeds the threshold of 550, the on-board LED (pin 13) turns ON, indicating a heartbeat.

SOURCE CODE

```
// Pulse Sensor Variables
int PulseSensorPurplePin = A0; // Pulse Sensor connected to Analog Pin A0
int LED13 = 13;                // On-board Arduino LED

int Signal;           // Holds raw pulse data (0 - 1024)
int Threshold = 550;  // Threshold for detecting heartbeat

void setup() {
  pinMode(LED13, OUTPUT); // Set LED pin as output
  Serial.begin(9600);     // Initialize serial communication
}

void loop() {
  Signal = analogRead(PulseSensorPurplePin); // Read Pulse Sensor data
  Serial.println(Signal);                    // Send signal value to Serial Plotter

  if (Signal > Threshold) { // If signal is above 550, turn ON LED
    digitalWrite(LED13, HIGH);
  } else {
    digitalWrite(LED13, LOW);  // Else, turn OFF LED
  }
  delay(10); // Small delay for stability
}
```

RESULT

The Pulse Sensor was successfully interfaced with NodeMCU ESP8266. Heartbeat signals are detected and displayed on the Serial Plotter; the LED blinks in sync with each detected heartbeat pulse.

Program 8: Sound Sensor

AIM

To interface a Sound Sensor module with an Arduino board and control an LED based on sound detection.

PROCEDURE

1. Connect Sound Sensor VCC to 5V on Arduino.
2. Connect Sound Sensor GND to GND on Arduino.
3. Connect Sound Sensor DO (Digital Output) to Pin 7 on Arduino.
4. Connect LED Anode (+) to Pin 13 on Arduino (built-in LED).
5. Connect LED Cathode (-) to GND on Arduino.
6. Open Arduino IDE and select the correct board and COM port.
7. Write and upload the given code.
8. Open Serial Monitor if needed.
9. When sound is detected (sensor output HIGH), the LED on Pin 13 turns ON.
10. When no sound is detected, the LED turns OFF.

SOURCE CODE

```
const int soundPin = 7;
const int ledPin   = 13;

void setup() {
  pinMode(soundPin, INPUT);
  pinMode(ledPin, OUTPUT);
}

void loop() {
  int soundState = digitalRead(soundPin);
  if (soundState == HIGH)
  {
    digitalWrite(ledPin, HIGH);
  } else
  {
    digitalWrite(ledPin, LOW);
  }
}
```

RESULT

The Sound Sensor was successfully interfaced with the Arduino board. The LED on Pin 13 turns ON when sound is detected and turns OFF when no sound is present.

Program 9: Ultrasonic Sensor

AIM

To interface an Ultrasonic Sensor (HC-SR04) with an Arduino board and measure the distance of an object by calculating the time taken for the echo signal to return.

PROCEDURE

1. Connect Ultrasonic Sensor VCC to 5V on Arduino.
2. Connect Ultrasonic Sensor GND to GND on Arduino.
3. Connect TRIG pin to Pin 9 on Arduino.
4. Connect ECHO pin to Pin 10 on Arduino.
5. Open Arduino IDE and select the correct board and COM port.
6. Write and upload the given code.
7. Open Serial Monitor at 9600 baud rate.
8. The TRIG pin sends a short ultrasonic pulse (10 microseconds).
9. The ECHO pin receives the reflected signal; duration is measured using pulseIn().
10. Distance is calculated using the formula: Distance = Duration x 0.034 / 2 (in cm).
11. The distance value is printed on the Serial Monitor every 500 ms.

SOURCE CODE

```
const int trigPin  = 9;
const int echoPin  = 10;
long duration;
int distance;

void setup()
{
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  Serial.begin(9600);
}

void loop()
{
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  duration = pulseIn(echoPin, HIGH);
```

```
  distance = duration * 0.034 / 2;
  Serial.print("Distance: ");
  Serial.print(distance);
  Serial.println(" cm");
  delay(500);
}
```

RESULT

The Ultrasonic Sensor was successfully interfaced with the Arduino board. The distance of objects is accurately measured and displayed in centimeters on the Serial Monitor.